C# OOP THEORETICAL QUESTIONS & ANSWERS

Comprehensive English Reference & Conceptual Guide

PART 1: OOP CORE CONCEPTS & INTERFACES

1. Why is it better to code against an interface rather than a concrete class?

Coding against an interface rather than a concrete class is a fundamental software design practice that offers key advantages:

- **Loose Coupling:** Reduces direct dependency on specific implementations, allowing components to evolve independently.
- **Flexibility & Maintainability:** Enables swapping underlying implementations at runtime without breaking consuming client code.
- **Testability:** Simplifies unit testing by allowing developers to inject mock objects or stubs instead of real dependencies.
- **Polymorphism:** Supports treating different object types uniformally as long as they implement the same interface.

2. When should you prefer an abstract class over an interface?

You should prefer an abstract class over an interface in the following scenarios:

- **Shared Code/Behavior:** When closely related classes need to share common, non-abstract implementation methods.
- **Internal State & Fields:** When you need to define fields, constants, or non-public members (protected, internal).
- **Strong "Is-A" Relationship:** When classes share a logical identity and common hierarchical base entity.
- **Version Control:** Adding new non-abstract members to an abstract class won't break derived classes.

3. How does implementing IComparable improve flexibility in sorting?

Implementing `IComparable` defines a default natural sorting order for custom objects. This integrates seamlessly with framework-provided sorting methods such as `Array.Sort()` or `List<T>.Sort()`, allowing collections of custom objects to be sorted automatically without writing explicit custom sorting logic every time.

4. What is the primary purpose of a copy constructor in C#?

The primary purpose of a copy constructor is to create a new object instance initialized with values cloned from an existing instance. It is mainly used to perform a **Deep Copy** of reference-type fields, ensuring that the new object maintains an independent state and changes to it do not affect the original object.

5. How does explicit interface implementation help in resolving naming conflicts?

Explicit interface implementation resolves naming conflicts when a class implements multiple interfaces containing methods with identical signatures, or when a class already defines a method with the same name as an interface method. By prefixing the method name with the interface name (e.g., `IWalkable.Walk()`), the method is hidden from the class's direct public contract and can only be invoked through a reference to that specific interface.

6. What is the key difference between encapsulation in structs and classes?

The primary difference stems from how memory and value assignment are handled:

- **Structs (Value Types):** Reside on the stack. Encapsulating state in a struct applies to that individual copy. Assigning or passing a struct creates a complete copy; modifying property values on a copy does not alter the original instance.
- **Classes (Reference Types):** Reside on the heap. Encapsulation protects shared state accessed via memory references; modifying properties through one reference affects all variables pointing to that same heap location.

7. How do default interface implementations affect backward compatibility in C#?

Default interface implementations allow developers to add new methods with default concrete code to existing interfaces without breaking legacy code. Older classes that implement the interface automatically inherit the default logic without failing compilation or requiring code updates.

8. How does constructor overloading improve class usability?

Constructor overloading enhances class usability by providing multiple object initialization options. Callers can instantiate an object with varying degrees of detailed parameters depending on available data, reducing the need for repetitive property setter chains after creation.

PART 2: ABSTRACTION, DESIGN PRINCIPLES & PATTERNS

1. What do we mean by coding against interface rather than class? And what do we mean by code against abstraction not concreteness?

- **Coding against interface rather than class:** Means designing client modules to interact with defined interface contracts rather than concrete class types. This decouples the caller from specific implementation details.
- **Coding against abstraction, not concreteness:** A broader design guideline (e.g., Dependency Inversion Principle) stating that high-level modules should depend on abstract entities (interfaces or abstract classes) rather than low-level concrete implementations, making systems modular, robust, and extensible.

2. What is abstraction as a guideline and how can we implement this through what we have studied?

Abstraction as a Guideline: Abstraction is the concept of focusing on essential characteristics and exposing *what* an object does while hiding the complex background details of *how* it achieves its work.

Implementation Methods in C#:

1. **Interfaces:** Creating pure abstraction contracts containing method/property signatures without implementation details.
2. **Abstract Classes:** Defining base entities with `abstract` methods that force derived classes to provide concrete implementations.
3. **Encapsulation:** Utilizing access modifiers (`private` , `protected`) to hide internal operational logic and data structures.